

C++ Codebook

This codebook contains commonly used algorithms and utilities optimized for competitive programming.

1. Basic Utilities

IO Speedup (cin, cout)

Accelerates C++ standard input/output operations. Avoid mixing cin/cout with scanf/printf after execution.

```
1 #include <iostream>
2
3 using namespace std;
4
5 void speedup() {
6     cin.sync_with_stdio(0);
7     cin.tie(0);
8 }
```

Greatest Common Divisor (GCD)

Calculates the greatest common divisor of two integers using the Euclidean algorithm.

```
1 int GCD(int x, int y) {
2     if (!x) return y;
3     if (!y) return x;
4     while ((x %= y) && (y %= x));
5     return x + y;
6 }
```

Least Common Multiple (LCM)

Calculates the least common multiple of two integers.

```
1 int LCM(int num1, int num2) {
2     return ((num1 * num2) / GCD(num1, num2));
3 }
```

Priority Queue (std::priority_queue)

Description: Demonstrates the use of C++ built-in std::priority_queue (from <queue>), including default behavior (Max-Heap), built-in Min-Heap, and Custom Struct Sorting. - **Default:** Max-Heap (largest element at the top). - **Built-in Min-Heap:** Uses std::greater (smallest element at the top). Very common with std::pair in graph algorithms. - **Custom Struct Sorting:** Overload the < operator inside a custom struct.

```
1 #include <queue>
2 #include <vector>
3 #include <iostream>
4 #include <utility> // for pair
5
6 using namespace std;
7
8 // Struct with custom sorting rules
9 struct Item {
```

```
10     int id;
11     int priority;
12
13     // Custom sorting rule: sort by priority ascending
14     ⇨ (min-heap)
15     // If priority is equal, sort by id ascending
16     ⇨ (smaller id at the top)
17     // Note: Since priority_queue is a Max-Heap by
18     ⇨ default, we invert the comparison logic.
19     bool operator<(const Item& other) const {
20         if (priority != other.priority) {
21             return priority > other.priority; // Larger
22             ⇨ priority means "less" priority in
23             ⇨ max-heap (so smaller comes to top)
24         }
25         return id > other.id; // Larger ID means "less"
26         ⇨ priority (so smaller ID comes to top)
27     }
28 };
29
30 void priorityQueueUsage() {
31     priority_queue<int> max_pq;
32     max_pq.push(10);
33     max_pq.push(30);
34     max_pq.push(20);
35     // max_pq.top() is 30 (largest element first)
36
37     priority_queue<int, vector<int>, greater<int>>
38     ⇨ min_pq;
39     min_pq.push(10);
40     min_pq.push(30);
41     min_pq.push(20);
42     // min_pq.top() is 10 (smallest element first)
43
44     priority_queue<pair<int, int>, vector<pair<int,
45     ⇨ int>>, greater<pair<int, int>>> pair_pq;
46     pair_pq.push({5, 101}); // {distance, node}
47     pair_pq.push({2, 102});
48     pair_pq.push({5, 100});
49     // pair_pq.top() is {2, 102} (smallest distance
50     ⇨ first, then smallest node index)
51
52     priority_queue<Item> custom_pq;
53     custom_pq.push({101, 5});
54     custom_pq.push({102, 5});
55     custom_pq.push({103, 2});
56     // custom_pq.top() is {103, 2} (priority 2), then
57     ⇨ {101, 5} (priority 5, smaller id first)
58 }
```

2. Prime Numbers

Count Primes (Sieve of Eratosthenes)

Description: Counts the number of prime numbers strictly less than or equal to N using the classical Sieve of Eratosthenes. **Time Complexity:** $O(N \log \log N)$ **Space Complexity:** $O(N)$ **Usage:** Efficiently counts large ranges of primes.

```
1 #include <vector>
2 #include <algorithm>
3
4 using namespace std;
5
```

```

6 int countPrimes(int N) {
7     if (N < 2) return 0;
8     vector<bool> isPrime(N + 1, true);
9     isPrime[0] = isPrime[1] = false;
10    for (int i = 2; i * i <= N; ++i) {
11        if (isPrime[i]) {
12            for (int j = i * i; j <= N; j += i) {
13                isPrime[j] = false;
14            }
15        }
16    }
17    return count(isPrime.begin(), isPrime.end(), true);
18 }

```

Prime Number Test

Description: Tests if a given 64-bit integer n is prime using $O(\sqrt{n})$ trial division optimized with a $6k \pm 1$ rule.

Time Complexity: $O(\sqrt{n})$

Space Complexity: $O(1)$ **Usage:** Fast primality test for single large integers.

```

1 bool isPrime(long long n) {
2     if (n <= 1) return false;
3     if (n <= 3) return true;
4     if (n % 2 == 0 || n % 3 == 0) return false;
5     for (long long i = 5; i * i <= n; i += 6) {
6         if (n % i == 0 || n % (i + 2) == 0) {
7             return false;
8         }
9     }
10    return true;
11 }

```

List Primes (Sieve of Eratosthenes)

Description: Generates and returns a list of all prime numbers less than or equal to N . **Time Complexity:** $O(N \log \log N)$

Space Complexity: $O(N)$

```

1 #include <vector>
2
3 using namespace std;
4
5 vector<int> sieve(int N) {
6     vector<bool> isPrime(N + 1, true);
7     isPrime[0] = isPrime[1] = false;
8     for (int i = 2; i * i <= N; ++i) {
9         if (isPrime[i]) {
10            for (int j = i * i; j <= N; j += i) {
11                isPrime[j] = false;
12            }
13        }
14    }
15    vector<int> primes;
16    for (int i = 2; i <= N; ++i) {
17        if (isPrime[i]) {
18            primes.push_back(i);
19        }
20    }
21    return primes;
22 }

```

Prime Factorization

Description: Performs prime factorization on a 64-bit integer n . **Time Complexity:** $O(\sqrt{n})$ **Space Complexity:** $O(\log n)$ (number of unique prime factors) **Returns:** A list of pairs where each pair is {base, exponent} (representing the base prime and its exponent/power).

```

1 #include <vector>
2 #include <utility>
3
4 using namespace std;
5
6 // Returns a vector of pairs: {prime_base, exponent}
7 vector<pair<long long, int>> primeFactorization(long
8     long n) {
9     vector<pair<long long, int>> factors;
10    for (long long i = 2; i * i <= n; ++i) {
11        if (n % i == 0) {
12            int count = 0;
13            while (n % i == 0) {
14                count++;
15                n /= i; // Eliminate the factor
16            }
17            factors.push_back({i, count});
18        }
19    }
20    if (n > 1) {
21        factors.push_back({n, 1}); // Remaining prime
22    }
23    return factors;
24 }

```

3. String Processing

String Split (Regex)

Description: Splits a string into substrings based on a regular expression pattern delimiter. **Parameters:** - **str:** The target string to split. - **keys:** Regex pattern specifying the delimiters (default matches dashes - or commas ,). - **keep:** If true, retains the matched delimiters as separate tokens in the output. **Usage:** Ideal for parsing complex string inputs. Requires <regex>.

```

1 #include <vector>
2 #include <string>
3 #include <regex>
4
5 using namespace std;
6
7 vector<string> split(const string& str, string keys =
8     "(-{1,3}|,)", bool keep = false) {
9     vector<string> tokens;
10    regex re(keys);
11    sregex_token_iterator it;
12    if (keep) {
13        it = sregex_token_iterator(str.begin(),
14            str.end(), re, {-1, 0});
15    } else {
16

```

```

14         it = sregex_token_iterator(str.begin(),
↪ str.end(), re, -1);
15     }
16     sregex_token_iterator end;
17     while (it != end) {
18         tokens.push_back(*it);
19         ++it;
20     }
21     return tokens;
22 }

```

Knuth-Morris-Pratt (KMP) Substring Search

Description: High-performance pattern searching algorithm. Precomputes the Longest Prefix Suffix (LPS) table to skip redundant character comparisons, eliminating the need to backtrack the text index. **Time Complexity:** $O(N + M)$ where N is text length, M is pattern length. **Space Complexity:** $O(M)$ for the LPS table. **Usage:** Fast substring matching when brute-force $O(N \cdot M)$ is too slow.

```

1  #include <vector>
2  #include <string>
3
4  using namespace std;
5
6  // Build the Longest Prefix which is also a Suffix
↪ (LPS) table
7  vector<int> buildLPS(const string &pattern) {
8      int m = pattern.size();
9      vector<int> lps(m, 0);
10     int len = 0; // Length of the previous longest
↪ prefix suffix
11     for (int i = 1; i < m; ) {
12         if (pattern[i] == pattern[len]) {
13             len++;
14             lps[i] = len;
15             i++;
16         } else {
17             if (len != 0) {
18                 len = lps[len - 1]; // Backtrack in
↪ pattern
19             } else {
20                 lps[i++] = 0;
21             }
22         }
23     }
24     return lps;
25 }
26
27 // KMP search main function: Returns starting indices
↪ of matches
28 vector<int> KMPsearch(const string &text, const string
↪ &pattern) {
29     vector<int> result;
30     int n = text.size(), m = pattern.size();
31     if (m == 0) return result;
32     vector<int> lps = buildLPS(pattern);
33     int i = 0, j = 0; // i -> text index, j -> pattern
↪ index
34     while (i < n) {
35         if (text[i] == pattern[j]) {
36             i++; j++;

```

```

37         if (j == m) {
38             result.push_back(i - j); // Found match
↪ at index (i - j)
39             j = lps[j - 1];
40         }
41     } else {
42         if (j != 0) {
43             j = lps[j - 1];
44         } else {
45             i++;
46         }
47     }
48 }
49 return result;
50 }

```

4. Computational Geometry

Closest Pair of Points (Divide & Conquer)

Description: Finds the closest pair of points in a 2D Euclidean plane using a divide-and-conquer strategy. 1. **Sort:** Sort all points by X-coordinates. 2. **Divide & Solve:** Recursively find the minimum distance in left and right halves. 3. **Combine:** Check points within a vertical strip of width $2d$ around the midline. Sort them by Y-coordinates and compare each point with at most 6 subsequent points. **Time Complexity:** $O(N \log N)$ **Space Complexity:** $O(N)$ **Usage:** Standard high-performance solution for closest-pair problems.

```

1  #include <vector>
2  #include <cmath>
3  #include <algorithm>
4  #include <limits>
5  #include <utility>
6
7  using namespace std;
8
9  struct Point {
10     double x, y;
11 };
12
13 // Calculate Euclidean distance between two points
14 double dist(const Point& a, const Point& b) {
15     return hypot(a.x - b.x, a.y - b.y);
16 }
17
18 // Brute force method for 3 or fewer points
19 double bruteForce(vector<Point>& points, int left, int
↪ right, pair<Point, Point>& bestPair) {
20     double minDist = numeric_limits<double>::max();
21     for (int i = left; i <= right; ++i) {
22         for (int j = i + 1; j <= right; ++j) {
23             double d = dist(points[i], points[j]);
24             if (d < minDist) {
25                 minDist = d;
26                 bestPair = {points[i], points[j]};
27             }
28         }
29     }
30     return minDist;
31 }

```

```

32 // Check the strip area for closer points crossing the
33 // divide boundary
34 double stripClosest(vector<Point>& strip, double d,
35     pair<Point, Point>& bestPair) {
36     double minDist = d;
37     sort(strip.begin(), strip.end(), [](const Point& a,
38     const Point& b) {
39         return a.y < b.y;
40     });
41     for (size_t i = 0; i < strip.size(); ++i) {
42         // Compare each point with next points inside
43         // the strip
44         for (size_t j = i + 1; j < strip.size() &&
45             (strip[j].y - strip[i].y) < minDist; ++j) {
46             double newDist = dist(strip[i], strip[j]);
47             if (newDist < minDist) {
48                 minDist = newDist;
49                 bestPair = {strip[i], strip[j]};
50             }
51         }
52     }
53     return minDist;
54 }
55
56 // Recursive Divide & Conquer helper utility
57 double closestUtil(vector<Point>& pointsSortedByX, int
58     left, int right, pair<Point, Point>& bestPair) {
59     // Use brute force if points count is 3 or less
60     if (right - left <= 3) {
61         return bruteForce(pointsSortedByX, left,
62             right, bestPair);
63     }
64     int mid = (left + right) / 2;
65     Point midPoint = pointsSortedByX[mid];
66     pair<Point, Point> leftPair, rightPair;
67     double dl = closestUtil(pointsSortedByX, left,
68         mid, leftPair);
69     double dr = closestUtil(pointsSortedByX, mid + 1,
70         right, rightPair);
71     double d = dl;
72     bestPair = leftPair;
73     if (dr < dl) {
74         d = dr;
75         bestPair = rightPair;
76     }
77     // Gather points inside vertical strip of width 2d
78     vector<Point> strip;
79     for (int i = left; i <= right; ++i) {
80         if (abs(pointsSortedByX[i].x - midPoint.x) < d)
81             strip.push_back(pointsSortedByX[i]);
82     }
83     pair<Point, Point> stripPair;
84     double dStrip = stripClosest(strip, d, stripPair);
85     if (dStrip < d) {
86         d = dStrip;
87         bestPair = stripPair;
88     }
89     return d;
90 }
91
92 // Main entry function to find the closest pair of
93 // points

```

```

86 pair<Point, Point> findClosestPair(vector<Point>&
87     points) {
88     vector<Point> pointsSortedByX = points;
89     sort(pointsSortedByX.begin(),
90     pointsSortedByX.end(), [](const Point& a, const
91     Point& b) {
92         return a.x < b.x;
93     });
94     pair<Point, Point> bestPair;
95     closestUtil(pointsSortedByX, 0,
96     pointsSortedByX.size() - 1, bestPair);
97     return bestPair;
98 }

```

5. Permutation Operations

Next Permutation (Array-based)

Description: Rearranges the array elements into the lexicographically next greater permutation of elements. If no such permutation exists (i.e., the array is sorted in descending order), it rearranges it into the lowest possible order (sorted in ascending order).

Time Complexity: $O(N)$

Space Complexity: $O(1)$ auxiliary space **Usage:** Finds the next permutation of a vector/array in-place or returns a copy of the next permutation.

```

1 #include <vector>
2 #include <algorithm>
3
4 using namespace std;
5
6 // Returns the lexicographically next permutation of
7 // the array.
8 vector<int> nextPermutation(vector<int> nums) {
9     int n = nums.size();
10    int i = n - 2;
11    // Find the first decreasing element from the right
12    while (i >= 0 && nums[i] >= nums[i + 1]) {
13        i--;
14    }
15    if (i >= 0) {
16        // Find the successor to nums[i] from the right
17        int j = n - 1;
18        while (nums[j] <= nums[i]) {
19            j--;
20        }
21        swap(nums[i], nums[j]);
22    }
23    // Reverse the remaining ascending suffix
24    reverse(nums.begin() + i + 1, nums.end());
25    return nums;
26 }

```

Permutation Rank (Lexicographical Rank)

Description: Calculates the 1-based lexicographical rank (index) of a permutation. This implementation handles generic values by using coordinate compression, uses a Fenwick Tree (Binary Indexed Tree) for $O(N \log N)$ complexity, and supports modulo arithmetic to prevent integer overflow for larger arrays.

Time Complexity: $O(N \log N)$

Space Complexity: $O(N)$ **Usage:** Given a permutation of unique elements, returns its 1-based rank modulo $10^9 + 7$.

```

1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Binary Indexed Tree (Fenwick Tree) helper
7  struct FenwickTree {
8      int n;
9      vector<int> tree;
10     FenwickTree(int n) : n(n), tree(n + 1, 0) {}
11     void update(int i, int delta) {
12         for (; i <= n; i += i & -i) tree[i] += delta;
13     }
14     int query(int i) {
15         int sum = 0;
16         for (; i > 0; i -= i & -i) sum += tree[i];
17         return sum;
18     }
19 };
20
21 // Calculates the 1-based lexicographical rank of a
22 // permutation.
23 // Assumes unique elements. Returns rank modulo MOD
24 // (default is 1e9 + 7).
25 long long getPermutationRank(vector<int> nums, long
26 // MOD = 1000000007) {
27     int n = nums.size();
28     if (n == 0) return 1;
29
30     // Precompute factorials modulo MOD
31     vector<long long> fact(n, 1);
32     for (int i = 1; i < n; ++i) {
33         fact[i] = (fact[i - 1] * i) % MOD;
34     }
35
36     // Coordinate compression to map values to [1, n]
37     vector<int> temp = nums;
38     sort(temp.begin(), temp.end());
39     vector<int> compressed(n);
40     for (int i = 0; i < n; ++i) {
41         compressed[i] = lower_bound(temp.begin(),
42 // temp.end(), nums[i]) - temp.begin() + 1;
43     }
44
45     FenwickTree bit(n);
46     for (int i = 1; i <= n; ++i) {
47         bit.update(i, 1);
48     }
49
50     long long rank = 1;
51     for (int i = 0; i < n; ++i) {
52         // Count how many unused elements to the right
53         // are smaller than compressed[i]
54         int smallerCount = bit.query(compressed[i] -
55 // 1);
56         rank = (rank + smallerCount * fact[n - 1 - i])
57 // % MOD;
58         // Mark current element as used
59         bit.update(compressed[i], -1);
60     }
61     return rank;
62 }

```

6. Math

Euler Totient Function

Description: Calculates Euler's totient function $\phi(x)$, which counts the number of positive integers strictly less than x that are coprime to x . **Time Complexity:** $O(\sqrt{x})$

Space Complexity: $O(1)$ **Usage:** Commonly used in number theory, particularly for finding modular multiplicative inverses and applying Euler's theorem.

```

1  #include <cmath>
2
3  int Phi(int x) {
4      if (x < 2) return 0;
5      int ret = x;
6      int sq = sqrt(x);
7      for (int p = 2; p <= sq; p++) {
8          if (x % p == 0) {
9              while (x % p == 0) x /= p;
10             ret -= ret / p;
11         }
12         if (x == 1) break;
13     }
14     if (x > 1) ret -= ret / x;
15     return ret;
16 }

```

7. Range Queries

Difference Array

Description: Efficiently applies multiple range addition updates $[L, R]$ by modifying only the boundaries of a difference array. A prefix sum is then used to reconstruct the final array values. **Returns:** The minimum coverage value across all positions after applying all updates. **Time Complexity:** $O(M + N)$ where M is the number of updates and N is the array size. **Space Complexity:** $O(N)$ for the difference array. **Usage:** Ideal for scenarios requiring many range updates followed by a single full-array query or state evaluation.

```

1  #include <iostream>
2  #include <vector>
3  #include <climits>
4
5  using namespace std;
6
7  long long differenceArray() {
8      ios::sync_with_stdio(false);
9      cin.tie(nullptr);
10
11     long long N, M;
12     if (!(cin >> N >> M)) return -1;
13
14     // Size N+2 to prevent R+1 from exceeding boundary
15     vector<long long> diff(N + 2, 0LL);
16
17     // Read interval [L, R] for each turret
18     for (int i = 0; i < M; i++) {
19         long long L, R;

```



```

20     cin >> L >> R;
21     // Difference array update: +1 at start L, -1
22     //   ↪ at position after end R+1
23     diff[L] += 1;
24     if (R + 1 <= N) {
25         diff[R + 1] -= 1;
26     }
27
28     // Compute prefix-sum directly on diff array and
29     //   ↪ track the minimum value
30     long long cover = 0; // Current cumulative
31     //   ↪ coverage at the current position
32     long long answer = LLONG_MAX; // Stores the minimum
33     //   ↪ coverage value
34
35     for (int pos = 1; pos <= N; pos++) {
36         cover += diff[pos]; // diff[pos] indicates the
37         //   ↪ change in coverage at pos
38         // cover represents the actual coverage count
39         //   ↪ at pos
40         if (cover < answer) {
41             answer = cover;
42         }
43     }
44
45     // Under normal circumstances, answer is at least
46     //   ↪ 0.
47     // Return the answer.
48     return answer;
49 }

```

8. Graph Algorithms

Minimum Spanning Tree (Kruskal's Algorithm)

Description: Finds the Minimum Spanning Tree (MST) of a connected, undirected, weighted graph. It uses the greedy approach: sorts all edges in non-decreasing order of their weight, and adds edges one by one if they don't form a cycle. A Disjoint Set Union (DSU) structure is used to check and manage cycles efficiently. **Time Complexity:** $O(E \log E + E\alpha(V))$ where E is the number of edges, V is the number of vertices, and α is the Inverse Ackermann function. **Space Complexity:** $O(V)$ for the DSU parent and rank arrays. **Usage:** Suitable for sparse graphs. Supports 1-indexed vertices by default.

```

1  #include <vector>
2  #include <algorithm>
3
4  using namespace std;
5
6  // Representation of an edge in the graph
7  struct Edge {
8      int u, v; // u, v: The two endpoint vertices
9      //   ↪ (nodes) connected by this edge
10     long long w; // w: The weight (cost/distance) of
11     //   ↪ this edge
12     // Overload the < operator to sort edges by weight
13     bool operator<(const Edge& other) const {
14         return w < other.w;
15     }
16 }

```

```

14 };
15
16 // Disjoint Set Union (DSU) / Union-Find structure
17 struct DSU {
18     vector<int> parent; // parent[i] stores the
19     //   ↪ parent/leader of element i
20     vector<int> rank; // rank[i] stores the
21     //   ↪ approximate depth of the tree rooted at i (for
22     //   ↪ Union by Rank optimization)
23
24     DSU(int n) { // n: The number of vertices/elements
25     //   ↪ in the graph
26         parent.resize(n + 1);
27         rank.resize(n + 1, 0);
28         for (int i = 0; i <= n; i++) {
29             parent[i] = i; // Initially, every vertex
30             //   ↪ is its own parent (disjoint set)
31         }
32     }
33
34     // Finds the representative (root leader) of the
35     //   ↪ set containing element i
36     int find(int i) {
37         if (parent[i] == i)
38             return i;
39         return parent[i] = find(parent[i]); // Path
40         //   ↪ compression optimization
41     }
42
43     // Merges the set containing element i with the set
44     //   ↪ containing element j.
45     // Returns true if they were in different sets and
46     //   ↪ successfully merged; false if they were already
47     //   ↪ in the same set.
48     bool unite(int i, int j) {
49         int root_i = find(i);
50         int root_j = find(j);
51         if (root_i != root_j) {
52             // Union by rank optimization: attach the
53             //   ↪ smaller tree under the larger tree
54             if (rank[root_i] < rank[root_j]) {
55                 parent[root_i] = root_j;
56             } else if (rank[root_i] > rank[root_j]) {
57                 parent[root_j] = root_i;
58             } else {
59                 parent[root_j] = root_i;
60                 rank[root_i]++;
61             }
62             return true; // Successfully merged
63         }
64         return false; // Already in the same set
65         //   ↪ (connecting them would form a cycle)
66     }
67 };
68
69 // Computes the MST weight and optionally stores the
70 //   ↪ selected edges.
71 // n: Number of vertices (nodes) in the graph.
72 // edges: Input vector containing all edges in the
73 //   ↪ graph (will be sorted in-place).
74 // mstEdges: Output vector to store the selected edges
75 //   ↪ that form the MST.
76 // Returns the total MST weight, or -1 if the graph is
77 //   ↪ disconnected (no MST exists).

```

```

62 long long kruskal(int n, vector<Edge>& edges,
63     ↪ vector<Edge>& mstEdges) {
64     mstEdges.clear();
65     sort(edges.begin(), edges.end()); // Sort edges in
66     ↪ ascending order of weight (greedy approach)
67
68     DSU dsu(n); // Disjoint Set Union to detect cycles
69     long long mst_weight = 0; // Accumulator for the
70     ↪ total weight of the MST
71     int edges_used = 0; // Counter for the number of
72     ↪ edges added to the MST
73
74     for (const auto& edge : edges) {
75         // Try to connect the two endpoints of the
76         ↪ current edge (edge.u and edge.v)
77         if (dsu.unite(edge.u, edge.v)) {
78             mst_weight += edge.w;
79             mstEdges.push_back(edge);
80             edges_used++;
81             // A tree with n vertices always has
82             ↪ exactly n - 1 edges
83             if (edges_used == n - 1) {
84                 break;
85             }
86         }
87     }
88
89     if (edges_used == n - 1) {
90         return mst_weight;
91     }
92     return -1; // Graph is not fully connected (could
93     ↪ not select n - 1 edges)
94 }

```